Lecture 12: Markov Decision Processes

Decision Making Under Uncertainty

Professor Anis Koubaa

SE 444
Alfaisal University

December 3, 2025

Outline

1. Introduction & Motivation
2. Dice Game Example
3. Markov Decision Process Definition
4. Policies & Value Functions
5. Optimal Policies & Value Iteration
6. Summary

Motivating Question

How would you get to Mountain View on Friday night in the least amount of time?

Options:

- Bike
- Drive
- Caltrain
- Uber/Lyft
- Fly

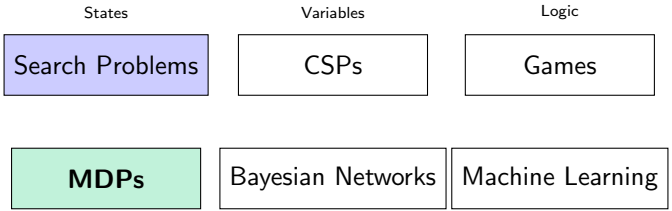
Challenges:

- Traffic is unpredictable
- Weather might change
- Delays are random
- Can't guarantee outcome

The Problem

We need to make decisions when outcomes are **uncertain**!

Course Plan: Where We Are



Today's Focus

Moving from deterministic search to stochastic decision-making

Search Problems: What We Know

Search Problem Framework

- **States:** Configurations of the world
- **Actions:** What we can do
- **Successor:** Succ(s, a) — deterministic next state
- **Cost:** Cost(s, a) — penalty for action
- **Goal:** Find minimum cost path

Key Assumption

Taking action a from state s deterministically results in a unique successor state Succ(s, a)

Problem: This assumption is unrealistic in the real world!

Uncertainty in the Real World

In reality, actions have random outcomes

Robotics:

- Actuators can fail
- Hit unseen obstacles
- Sensors are noisy
- Movement is imprecise

Agriculture:

- Unknown weather
- Variable crop yield
- Pest outbreaks
- Market prices

Resource Allocation:

- Unknown customer demand
- Market fluctuations
- Supply chain delays
- Competitor actions

Games:

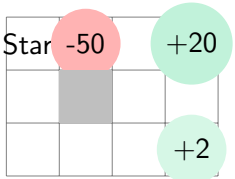
- Dice rolls
- Card shuffles
- Opponent strategies
- Random events

Example: Volcano Crossing

Exploring a South Pacific Island (3×4 grid)

Setup:

- Start at (2,1)
- Actions: N, E, S, W
- Lava lake: -50
- Safe area: +2
- Fabulous view: +20



Slippery floor!

The Twist:

- Treacherous terrain!
- Probability of slipping
- Move in random direction

Question:

- Low slip: Go for +20? ✓
- High slip: Play safe +2? ✓

Example: Dice Game

Game Rules

For each round $r = 1, 2, \ldots$:

1. You choose **stay** or **quit**
2. If **quit**: Get \$10 and game ends
3. If **stay**: Get \$4, then roll a 6-sided die
   - If die shows 1 or 2: Game ends
   - Otherwise: Continue to next round

What is the best strategy?

Dice Game: "Stay" Policy

If you always stay:

Possible Outcomes:

- Round 1 ends: $\frac{1}{3}$ chance, get \$4
- Round 2 ends: $\frac{2}{3} \cdot \frac{1}{3}$ chance, get \$8
- Round 3 ends: $(\frac{2}{3})^2 \cdot \frac{1}{3}$ chance, get \$12
- And so on...

Expected Utility:

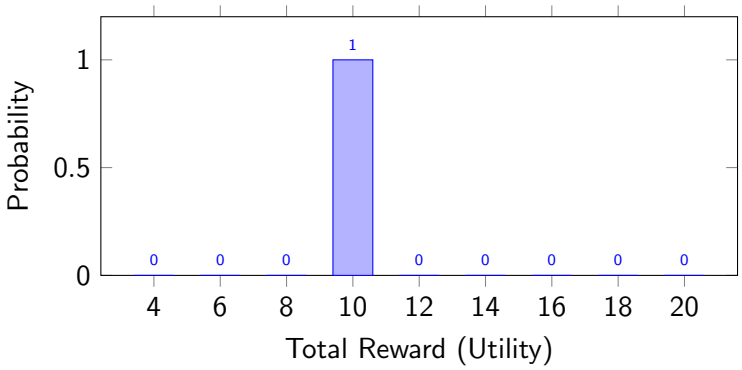
$$\begin{aligned}\mathbb{E}[U] &= \frac{1}{3}(4) + \frac{2}{3} \cdot \frac{1}{3}(8) \\ &\quad + \frac{2}{3} \cdot \frac{2}{3} \cdot \frac{1}{3}(12) + \cdots \\ &= \mathbf{12}\end{aligned}$$

Key Insight

Even though outcomes vary, the **expected utility** is 12

Dice Game: "Quit" Policy

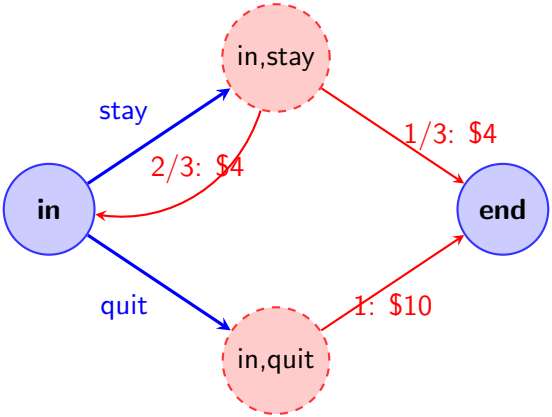
If you always quit:



Expected Utility

$$\mathbb{E}[U] = 1 \times 10 = \mathbf{10}$$

MDP Graph for Dice Game



MDP Components

- States (circles): in, end
- Actions (blue edges): stay, quit

Markov Decision Process (MDP)

Formal Definition

An MDP is defined by:

- **States**: Set of all possible states
- $s_{\text{start}} \in$ States: Starting state
- **Actions**(s): Possible actions from state s
- $T(s, a, s')$: Probability of reaching s' if take action a in state s
- **Reward**(s, a, s'): Immediate reward for transition (s, a, s')
- **IsEnd**(s): Whether s is a terminal state
- $0 \leq \gamma \leq 1$: Discount factor (default: 1)

Key Difference from Search

Succ$(s, a) \Rightarrow T(s, a, s')$ — From deterministic to probabilistic!

Transition Probabilities

Definition

$T(s, a, s')$ specifies the probability of ending up in state s' if we take action a in state s

Key Property: For each state s and action a:

$$\sum_{s' \in \text{States}} T(s, a, s') = 1$$

Dice Game Transitions

s	a	s'	$T(s, a, s')$
in	quit	end	1
in	stay	in	2/3
in	stay	end	1/3

Search Problems vs MDPs

Search Problem:

- States
- s_{start}
- Actions(s)
- Succ(s, a) — deterministic
- Cost(s, a) — minimize
- IsEnd(s)

Goal:

- Find minimum cost path
- Single sequence of actions

MDP:

- States
- s_{start}
- Actions(s)
- $T(s, a, s')$ — probabilistic
- Reward(s, a, s') — maximize
- IsEnd(s)
- Discount γ

Goal:

- Find optimal policy
- Mapping: state $\rightarrow$ action

What is a Policy?

Definition: Policy

A **policy** π is a function that maps each state to an action:

$$\pi : \text{States} \rightarrow \text{Actions}$$

$$\pi(s) = \text{action to take in state } s$$

Dice Game Policies

- **Policy 1:** $\pi_1(in) = \text{stay}$ — Always stay
- **Policy 2:** $\pi_2(in) = \text{quit}$ — Always quit
- **Policy 3:** $\pi_3(in) = \text{random}$ — Flip a coin

Key Insight

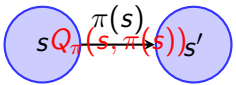
Value of a Policy

Definition: Value Function

Let $V_\pi(s)$ be the **expected utility** of following policy π starting from state s

Definition: Q-Value

Let $Q_\pi(s, a)$ be the expected utility of taking action a from state s, then following policy π



$V_\pi(s)$

Bellman Equations for Policy Evaluation

Value Function Recurrence

$$V_\pi(s) = \begin{cases} 0 & \text{if IsEnd}(s) \\ Q_\pi(s, \pi(s)) & \text{otherwise} \end{cases}$$

Q-Value Recurrence

$$Q_\pi(s, a) = \sum_{s'} T(s, a, s') \left[\text{Reward}(s, a, s') + \gamma V_\pi(s')\right]$$

Interpretation:

- $V_\pi(s)$: Follow the policy's action from s
- $Q_\pi(s, a)$: Take action a, then follow policy
- Expectation over all possible next states s'
- Immediate reward + discounted future value

Policy Evaluation Algorithm

Goal

Given a policy π, compute $V_\pi(s)$ for all states s

Algorithm: Iterative Policy Evaluation

1. Initialize $V_\pi^{(0)}(s) \leftarrow 0$ for all states s
2. For iteration $t = 1, \ldots, t_{PE}$:
   - For each state s:

$$V_\pi^{(t)}(s) \leftarrow \sum_{s'} T(s, \pi(s), s')[\text{Reward}(s, \pi(s), s') + \gamma V_\pi^{(t-1)}(s')]$$

3. Repeat until convergence: $\max_s |V_\pi^{(t)}(s) - V_\pi^{(t-1)}(s)| \leq \epsilon$

Time Complexity: $O(t_{PE} \cdot S \cdot S')$ where S = number of states

Optimal Value and Policy

Definition: Optimal Value

The **optimal value** $V_{\text{opt}}(s)$ is the maximum value attained by any policy:

$$V_{\text{opt}}(s) = \max_{\pi} V_{\pi}(s)$$

Definition: Optimal Q-Value

$$Q_{\text{opt}}(s, a) = \sum_{s'} T(s, a, s')[\text{Reward}(s, a, s') + \gamma V_{\text{opt}}(s')]$$

Bellman Optimality Equation

$$V_{\text{opt}}(s) = \begin{cases} 0 & \text{if IsEnd}(s) \\ \max_{a \in \text{Actions}(s)} Q_{\text{opt}}(s, a) & \text{otherwise} \end{cases}$$

Extracting the Optimal Policy

Optimal Policy

Once we have $V_{\text{opt}}(s)$ or $Q_{\text{opt}}(s, a)$, the optimal policy is:

$$\pi_{\text{opt}}(s) = \arg \max_{a \in \text{Actions}(s)} Q_{\text{opt}}(s, a)$$

Intuition:

- $Q_{\text{opt}}(s, a)$ tells us the value of taking action a from state s
- The optimal action is simply the one with the highest Q-value
- Choose the action that maximizes expected future reward

Important

The optimal policy is **deterministic** — always take the best action!

Value Iteration Algorithm

Algorithm: Value Iteration [Bellman, 1957]

1. Initialize $V_{\text{opt}}^{(0)}(s) \leftarrow 0$ for all states s
2. For iteration $t = 1, \ldots, t_{\text{VI}}$:
   - For each state s:

$$V_{\text{opt}}^{(t)}(s) \leftarrow \max_{a \in \text{Actions}(s)} \sum_{s'} T(s, a, s') \times [\text{Reward}(s, a, s') + \gamma V_{\text{opt}}^{(t-1)}(s')]$$

3. Repeat until convergence

Time Complexity: $O(t_{\text{VI}} \cdot S \cdot A \cdot S')$

Output: $V_{\text{opt}}(s)$ and $\pi_{\text{opt}}(s) = \arg\max_a Q_{\text{opt}}(s, a)$

Value Iteration: Intuition

Value iteration is dynamic programming for MDPs

Iteration 0:

- All values = 0
- No information

Iteration 1:

- Values = immediate rewards
- 1-step lookahead

Iteration 2:

- 2-step lookahead
- Better estimates

Iteration t:

- t-step lookahead
- Optimal for t steps

Convergence:

- Values stabilize
- $V_{\text{opt}}^{(t)} \approx V_{\text{opt}}^{(t-1)}$
- Found optimal policy!

Key Insight

Convergence Conditions

When Does Value Iteration Converge?

Value iteration is guaranteed to converge if **either**:

1. Discount factor $\gamma < 1$, OR
2. The MDP graph is **acyclic**

Interpretation of $\gamma < 1$:

- With probability $(1 - \gamma)$, the MDP terminates at any state
- Equivalent to adding a transition to a special end state
- Ensures finite expected path length

Warning

If $\gamma = 1$ AND the graph has cycles, value iteration may not converge!

Summary of Algorithms

Two Key Algorithms

1. **Policy Evaluation:** $(\text{MDP}, \pi) \to V_\pi$
   - Given a policy, compute its value
   - Iteratively apply Bellman equations
   - Time: $O(t_{PE} \cdot S \cdot S')$
2. **Value Iteration:** $\text{MDP} \to (V_{\text{opt}}, \pi_{\text{opt}})$
   - Find the optimal policy
   - Iteratively compute optimal values
   - Time: $O(t_{\text{VI}} \cdot S \cdot A \cdot S')$

Unifying Idea: Write recurrence $\to$ turn into iterative algorithm

Key Takeaways

1. **MDPs model uncertainty**
   - Actions have probabilistic outcomes
   - Transition function $T(s, a, s')$ instead of $\text{Succ}(s, a)$
2. **Solutions are policies, not paths**
   - Policy π: state $\rightarrow$ action mapping
   - Handles all possible outcomes
3. **Maximize expected utility**
   - Can't guarantee outcomes
   - Optimize expected value instead
4. **Bellman equations are fundamental**
   - Recursive structure of value functions
   - Lead directly to algorithms

Next Steps

What's Next?

- **Reinforcement Learning:**
  - What if we don't know $T(s, a, s')$?
  - What if we don't know Reward(s, a, s')?
  - Learn from experience!
- **Policy Iteration:**
  - Alternative to value iteration
  - Directly improve policies
  - Often faster convergence
- **Partially Observable MDPs (POMDPs):**
  - Can't fully observe the state
  - Maintain belief distributions
  - Even more realistic!

Thank You!

Questions?

SE 444: Artificial Intelligence
Lecture 12: Markov Decision Processes
Professor Anis Koubaa